

Tema Algoritmo DDA

Docente: Richard Sivila

Algoritmo DDA

Primitivas gráficas con raylib

Las **primitivas gráficas** son los elementos más básicos con los que se construyen imágenes en computación gráfica. En un entorno 2D, las más importantes al inicio son:

- punto
- línea
- rectángulo
- circunferencia
- polígonos

En **raylib**, estas primitivas ya vienen implementadas mediante funciones sencillas como:

- DrawPixel()
- DrawLine()
- DrawRectangle()
- DrawCircle()

3.1 Sistemas de coordenadas de referencia

Un sistema de coordenadas permite ubicar objetos dentro de una superficie de dibujo.

a) Coordenadas de pantalla

En **raylib**, al igual que en muchas librerías modernas 2D, el sistema de coordenadas de pantalla funciona así:

- el origen (0,0) está en la **esquina superior izquierda**
- el eje **X** crece hacia la **derecha**
- el eje **Y** crece hacia **abajo**

Esquema

(0,0)----->X
|
|
|
v
Y

Si la ventana mide 800 x 600, entonces:

- esquina superior izquierda: (0,0)
- esquina superior derecha: (799,0)

- esquina inferior izquierda: (0,599)
- esquina inferior derecha: (799,599)

Ejemplo en raylib

```
#include "raylib.h"

int main() {
    InitWindow(800, 600, "Coordenadas de pantalla");

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);
        DrawText("(0,0)", 10, 10, 20, BLACK);
        DrawCircle(0, 0, 5, RED);
        DrawText("(400,300)", 410, 300, 20, BLUE);
        DrawCircle(400, 300, 5, BLUE);

        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

b) Especificaciones absolutas y relativas de coordenadas

Coordenadas absolutas

Indican una posición exacta respecto al origen global de la pantalla.

Ejemplo:

```
DrawCircle(200, 150, 4, RED);
```

Aquí el punto se dibuja exactamente en (200,150).

Coordenadas relativas

Se expresan a partir de otra posición conocida.

Ejemplo:

Si un objeto está en (200,150) y quieres moverlo 30 unidades a la derecha y 20 hacia abajo:

- nueva posición = (230,170)

En código:

```
int x = 200;
int y = 150;

x = x + 30;
y = y + 20;
DrawCircle(x, y, 4, RED);
```

Idea clave

- **absoluta:** “dibuja en este lugar exacto”
- **relativa:** “muévete desde donde ya estás”

Esto es muy útil en videojuegos para:

- movimiento de personajes
- desplazamiento de cámaras
- trayectorias
- animaciones

3.2 Especificación de un sistema bidimensional de referencia universal y OpenGL

En los textos clásicos de gráficos, primero se define un **sistema de referencia universal** o de mundo. Luego ese sistema se transforma al sistema de pantalla.

Idea teórica

Se suele trabajar con dos espacios:

1. Coordenadas del mundo

Representan posiciones lógicas dentro de la escena.

Ejemplo:

- jugador en (10,5)
- enemigo en (20,8)

2. Coordenadas de pantalla

Son las posiciones reales en píxeles donde finalmente se dibujan los objetos.

En OpenGL clásico

Normalmente se definía una región visible mediante proyecciones ortográficas, por ejemplo con funciones como:

- `glOrtho(...)`
- `gluOrtho2D(...)`

Eso permitía decir: “quiero que mi mundo 2D vaya de x_{min} a x_{max} y de y_{min} a y_{max} ”.

En raylib

Raylib simplifica bastante esta idea. En 2D ya trabaja sobre coordenadas de pantalla, pero se puede simular un sistema universal usando **Camera2D**.

Ejemplo conceptual

```
#include "raylib.h"

int main() {
    InitWindow(800, 600, "Sistema 2D con Camera2D");

    Camera2D cam = {0};
    cam.target = (Vector2){0, 0};
    cam.offset = (Vector2){400, 300};
    cam.zoom = 20.0f;

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);

        BeginMode2D(cam);

        DrawLine(-100, 0, 100, 0, RED); // eje X
        DrawLine(0, -100, 0, 100, BLUE); // eje Y
        DrawCircle(10, 5, 1, GREEN);

        EndMode2D();

        DrawText("Sistema de referencia universal con Camera2D", 10, 10, 20, BLACK);
        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

Interpretación

Aquí se construye un sistema más parecido al matemático:

- el centro de pantalla representa el origen
- se puede hacer zoom
- los objetos se dibujan en coordenadas del “mundo”

Esto se acerca bastante al enfoque teórico usado en OpenGL y en los libros clásicos.

3.3 Funciones de punto en OpenGL y su equivalente en raylib

En OpenGL

Tradicionalmente, para dibujar puntos se usaban funciones como:

- glBegin(GL_POINTS)
- glVertex2i(x,y)
- glEnd()

También se podía controlar el tamaño del punto con:

- glPointSize()

Equivalente en raylib

En raylib se usa:

DrawPixel(x, y, color);

o también, si se quiere visualizar mejor:

DrawCircle(x, y, 2, color);

Ejemplo

```
#include "raylib.h"

int main() {
    InitWindow(800, 600, "Dibujo de puntos");

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);

        DrawPixel(100, 100, RED);
        DrawPixel(101, 100, RED);
        DrawPixel(102, 100, RED);

        DrawCircle(200, 200, 3, BLUE);

        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

Idea didáctica

En raylib ya dibuja puntos directamente, en una materia de programación gráfica conviene explicar que:

- una línea no es más que una secuencia de puntos
- un polígono está formado por líneas
- una imagen raster está compuesta por píxeles

3.4 Funciones OpenGL para líneas y su equivalente en raylib

En OpenGL clásico

Se usaban funciones como:

- glBegin(GL_LINES)
- glVertex2i(x1,y1)
- glVertex2i(x2,y2)
- glEnd()

o GL_LINE_STRIP, GL_LINE_LOOP.

En raylib

La función más simple es:

DrawLine(x1, y1, x2, y2, color);

También existen:

- DrawLineV()
- DrawLineEx()
- DrawLineBezier()

Ejemplo

```
#include "raylib.h"

int main() {
    InitWindow(800, 600, "Lineas con raylib");

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);

        DrawLine(100, 100, 300, 100, RED);
        DrawLine(100, 100, 100, 300, BLUE);
        DrawLine(100, 100, 300, 300, GREEN);

        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

3.5 Algoritmos de dibujo de líneas

En gráficos por computadora, un problema fundamental es: **dado un punto inicial y uno final, cómo decidir qué píxeles encender para formar una línea lo más continua posible.**

Los textos clásicos presentan varios enfoques, entre ellos:

- ecuación de la recta
- algoritmo DDA
- algoritmo de Bresenham

Aquí trabajaremos con:

- ecuación de la línea
- algoritmo DDA

a) Ecuaciones de las líneas

La forma pendiente-intersección es:

$$y = mx + b$$

donde:

- m = pendiente
- b = intersección con el eje Y

La pendiente se calcula como:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Problema práctico

Esta fórmula es útil para la teoría, pero para dibujar en pantalla tiene limitaciones:

- si la línea es vertical, la pendiente no está definida
- usar números reales en cada píxel puede ser costoso
- se requiere redondear continuamente

Por eso aparecen algoritmos incrementales como **DDA**.

Algoritmo DDA

DDA significa **Digital Differential Analyzer**.

La idea es muy simple:

1. tomar los extremos de la línea
2. calcular cuánto cambia x y cuánto cambia y
3. dividir el recorrido en pasos pequeños
4. en cada paso dibujar un píxel aproximado

Fórmulas básicas

Dados dos puntos:

$$(x_1, y_1), (x_2, y_2)$$

se calcula:

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

El número de pasos será:

$$steps = \max(|dx|, |dy|)$$

Luego los incrementos por paso son:

$$xInc = \frac{dx}{steps}$$

$$yInc = \frac{dy}{steps}$$

Y se genera la línea con:

- `x = x + xInc`
- `y = y + yInc`

dibujando el punto redondeado en cada iteración.

Ejemplo numérico

Queremos unir:

- $P1 = (2,2)$
- $P2 = (10,6)$

Entonces:

$$dx = 10 - 2 = 8$$

$$dy = 6 - 2 = 4$$

$$steps = \max(8, 4) = 8$$

$$xInc = 8/8 = 1$$

$$yInc = 4/8 = 0.5$$

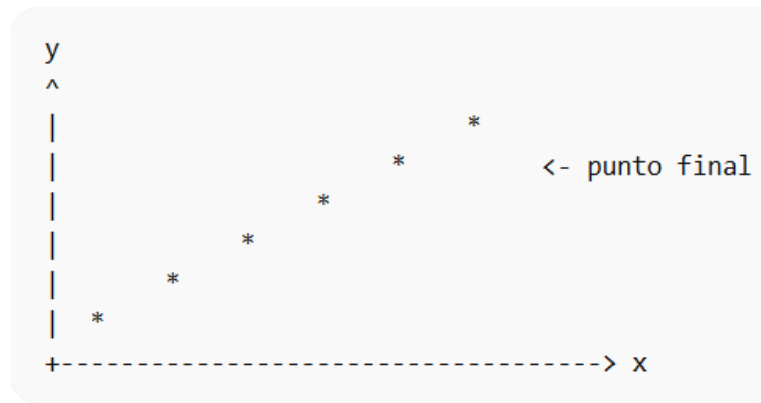
Ahora generamos los puntos:

Paso	x	y	píxel dibujado
0	2.0	2.0	(2,2)
1	3.0	2.5	(3,3)
2	4.0	3.0	(4,3)
3	5.0	3.5	(5,4)
4	6.0	4.0	(6,4)
5	7.0	4.5	(7,5)
6	8.0	5.0	(8,5)
7	9.0	5.5	(9,6)
8	10.0	6.0	(10,6)

Gráfico explicativo del DDA

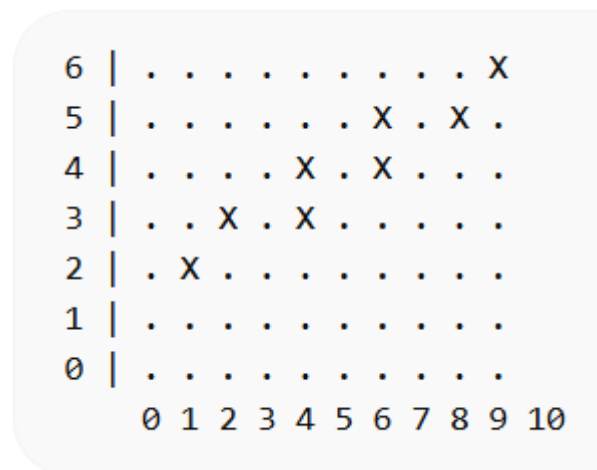
Línea ideal vs píxeles rasterizados

Los asteriscos representan los píxeles aproximados que se van activando.



Visualización en rejilla

Para una línea de (2,2) a (10,6):



No todos los puntos son exactos en valor real; se redondean al píxel más cercano.

Ventajas del DDA

- fácil de entender
- bueno para enseñanza inicial
- muestra claramente el concepto de interpolación

Desventajas

- usa números reales
- requiere redondeo
- puede acumular error

Por eso, posteriormente suele enseñarse **Bresenham**, que trabaja mejor con enteros.

Implementación del algoritmo DDA en raylib

```

#include "raylib.h"
#include <cmath>

void DrawLineDDA(int x1, int y1, int x2, int y2, Color color) {
    int dx = x2 - x1;
    int dy = y2 - y1;

    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);

    float xInc = dx / (float)steps;
    float yInc = dy / (float)steps;

    float x = (float)x1;
    float y = (float)y1;

    for (int k = 0; k <= steps; k++) {
        DrawPixel((int)round(x), (int)round(y), color);
        x += xInc;
        y += yInc;
    }
}

int main() {
    InitWindow(800, 600, "Algoritmo DDA con raylib");

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);
        // ejes de referencia
        DrawLine(50, 500, 750, 500, LIGHTGRAY);
        DrawLine(50, 100, 50, 500, LIGHTGRAY);
        // línea dibujada con DDA
        DrawLineDDA(100, 400, 500, 200, RED);
        // extremos
        DrawCircle(100, 400, 4, BLUE);
        DrawCircle(500, 200, 4, BLUE);
        DrawText("Linea generada con DDA", 520, 200, 20, BLACK);
        EndDrawing();
    }
    CloseWindow();
    return 0;
}

```

Versión didáctica: mostrar cada píxel como un cuadrito

Para que el estudiante vea mejor cómo trabaja DDA, conviene dibujar una malla y representar cada píxel como una celda ampliada.

```

#include "raylib.h"
#include <cmath>

const int CELL = 40;

void DrawGrid(int cols, int rows, int offsetX, int offsetY) {
    for (int i = 0; i <= cols; i++) {
        DrawLine(offsetX + i * CELL, offsetY, offsetX + i * CELL, offsetY + rows * CELL,
LIGHTGRAY);
    }
    for (int j = 0; j <= rows; j++) {
        DrawLine(offsetX, offsetY + j * CELL, offsetX + cols * CELL, offsetY + j * CELL,
LIGHTGRAY);
    }
}

void FillCell(int x, int y, int offsetX, int offsetY, Color color) {
    DrawRectangle(offsetX + x * CELL + 1, offsetY + y * CELL + 1, CELL - 2, CELL - 2, color);
}

void DrawLineDDA_Grid(int x1, int y1, int x2, int y2, int offsetX, int offsetY) {
    int dx = x2 - x1;
    int dy = y2 - y1;

    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);

    float xInc = dx / (float)steps;
    float yInc = dy / (float)steps;

    float x = (float)x1;
    float y = (float)y1;

    for (int k = 0; k <= steps; k++) {
        FillCell((int)round(x), (int)round(y), offsetX, offsetY, ORANGE);
        x += xInc;
        y += yInc;
    }
}

int main() {
    InitWindow(900, 700, "DDA en rejilla");

    while (!WindowShouldClose()) {
        BeginDrawing();
        ClearBackground(RAYWHITE);

        int offsetX = 80;
        int offsetY = 80;

        DrawGrid(15, 10, offsetX, offsetY);
        DrawLineDDA_Grid(2, 2, 10, 6, offsetX, offsetY);

        DrawText("Visualizacion ampliada del algoritmo DDA", 80, 30, 24, BLACK);

        EndDrawing();
    }

    CloseWindow();
    return 0;
}

```

Esto es muy útil en clase porque permite ver claramente:

- la malla
- los pasos
- los píxeles activados
- la aproximación de la recta

Conclusión

Adaptando estos temas al entorno de **raylib**, se puede enseñar informática gráfica inicial de forma clara y práctica:

- las **coordenadas de pantalla** explican cómo se localizan los objetos
- las **coordenadas absolutas y relativas** ayudan a comprender movimiento y posicionamiento
- las primitivas básicas como **puntos y líneas** tienen equivalentes directos en raylib
- el **algoritmo DDA** sigue siendo fundamental para entender cómo una línea continua se representa con píxeles discretos

Desde el punto de vista didáctico, raylib es muy conveniente porque permite:

- implementar los conceptos clásicos
- visualizar resultados rápido
- relacionar la teoría con videojuegos 2D

Ejercicio Propuesto

Tema: Primitivas gráficas y algoritmo DDA con raylib

Objetivo

Desarrollar una aplicación en **C/C++ con raylib** que permita:

1. Comprender el sistema de coordenadas de pantalla
2. Diferenciar coordenadas absolutas y relativas
3. Implementar el algoritmo **DDA**
4. Visualizar cómo una línea se construye píxel a píxel

Enunciado

Desarrollar un programa que:

1. Dibuje una **rejilla (grid)** que represente píxeles ampliados
2. Permita definir dos puntos:
 - Punto inicial (x1, y1)
 - Punto final (x2, y2)
3. Dibuje la línea usando:
 - Función nativa de raylib (DrawLine)
 - Algoritmo implementado manualmente (**DDA**)
4. Muestre ambos resultados para comparación

5. Permita mover los puntos con teclado

Requerimientos funcionales

El programa debe:

1. Sistema de coordenadas

- Mostrar una rejilla (mínimo 15x10 celdas)
- Cada celda representa un “píxel”

2. Entrada de datos (interacción)

Usar teclado para mover los puntos:

Tecla	Acción
W A S D	mover punto inicial
Flechas	mover punto final

3. Visualización

Debe mostrar:

- puntos inicial y final (en color azul)
- línea con DDA (color rojo)
- línea con función raylib (color verde)
- coordenadas en pantalla

4. Comparación

Se debe observar:

- cómo el DDA genera la línea paso a paso
- diferencia (si existe) con DrawLine

Preguntas de análisis (para el estudiante)

1. ¿Qué diferencia observas entre la línea generada por DDA y la de raylib?
2. ¿Por qué se usa $\text{steps} = \max(|dx|, |dy|)$?
3. ¿Qué ocurre si la línea es vertical?
4. ¿Qué error introduce el redondeo en DDA?
5. ¿Cómo cambiaría el algoritmo si trabajáramos solo con enteros?

Extensiones (nivel avanzado)

El estudiante puede mejorar el ejercicio:

- implementar algoritmo de **Bresenham** (sacar al azar un estudiante, para que explique)

- animar el trazado paso a paso
- cambiar escala del grid

Criterios de evaluación

Criterio	Puntos
implementación DDA	30
uso correcto de coordenadas	20
interacción (teclado)	20
visualización clara	20
análisis teórico	10

Resultado esperado

El estudiante comprenderá:

- cómo funciona un sistema de coordenadas en pantalla
- cómo una línea se discretiza en píxeles
- cómo implementar un algoritmo clásico de gráficos
- la diferencia entre teoría y librerías modernas